

Cooling Solver

Compilazione su Leonardo

Aggiornata al 18 settembre 2026

1 Scopo

Questa guida descrive la compilazione manuale del Cooling Solver sul cluster Leonardo. Sono mantenuti due profili distinti:

Confronto tra implementazioni produce file HDF5 da validare e usa opzioni floating-point conservative per ridurre le differenze numeriche;

Benchmark dello speed-up elimina completamente HDF5 e abilita le ottimizzazioni più aggressive appropriate per la CPU Ice Lake e la GPU NVIDIA A100 di Leonardo.

I due profili producono eseguibili con nomi diversi, per evitare di usare per errore una build di validazione durante un benchmark. Tutti i comandi devono essere lanciati dalla directory principale del progetto.

2 Ambiente software

2.1 Moduli comuni

Iniziare da un ambiente riproducibile e caricare NVHPC:

```
module purge
module load nvhpc

mkdir -p install/bin
```

2.2 Modulo aggiuntivo per il confronto HDF5

Solo per il profilo di confronto è necessario caricare HDF5:

```
module load hdf5/1.14.3--gcc--12.2.0--spack0.22

HDF5_PREFIX="$(dirname "$(dirname "$(command -v h5c++))")"
HDF5_INCLUDE="${HDF5_PREFIX}/include"
HDF5_LIB="${HDF5_PREFIX}/lib"
```

3 Profilo 1: confronto tra implementazioni

3.1 Obiettivo del profilo

Questo profilo serve a confrontare i campi prodotti dalle quattro implementazioni. Tutti gli eseguibili:

- sono compilati con lo stesso compilatore `nvc++`;
- supportano la scrittura HDF5;
- usano doppia precisione e opzioni floating-point conservative;
- mantengono `-O3`, perché il confronto deve comunque essere rappresentativo di una normale build ottimizzata.

Queste opzioni riducono alcune fonti di differenza, ma non garantiscono uguaglianza bit-a-bit: l'ordine delle operazioni e delle riduzioni cambia tra seriale, OpenMP, OpenACC multicore e GPU.

3.2 CPU seriale con HDF5

```
nvc++ -O3 -std=c++17 \
-Kieee \
-Mnofma \
-DUSE_HDF5=1 \
-I"${HDF5_INCLUDE}" \
src/cpp/cooling.cpp \
-o install/bin/cooling_cpp_compare \
-L"${HDF5_LIB}" \
-Wl,-rpath,"${HDF5_LIB}" \
-lhdf5_cpp -lhdf5 -lz -ldl -lm -lstdc++fs
```

3.3 CPU OpenMP con HDF5

```
nvc++ -O3 -std=c++17 \
-mp \
-Kieee \
-Mnofma \
-DUSE_HDF5=1 \
-I"${HDF5_INCLUDE}" \
src/cpp/cooling_openmp.cpp \
-o install/bin/cooling_omp_compare \
-L"${HDF5_LIB}" \
-Wl,-rpath,"${HDF5_LIB}" \
-lhdf5_cpp -lhdf5 -lz -ldl -lm -lstdc++fs
```

3.4 CPU OpenACC multicore con HDF5

```
nvc++ -O3 -std=c++17 \
-acc=multicore \
-Minfo=accel \
-Kieee \
-Mnofma \
-DUSE_HDF5=1 \
-I"${HDF5_INCLUDE}" \
src/cpp/cooling_openacc.cpp \
-o install/bin/cooling_acc_cpu_compare \
-L"${HDF5_LIB}" \
-Wl,-rpath,"${HDF5_LIB}" \
-lhdf5_cpp -lhdf5 -lz -ldl -lm -lstdc++fs
```

3.5 GPU OpenACC A100 con HDF5

```
nvc++ -O3 -std=c++17 \  
-acc=gpu \  
-gpu=cc80,nofma \  
-Minfo=accel \  
-Kieee \  
-Mnofma \  
-DUSE_HDF5=1 \  
-I"${HDF5_INCLUDE}" \  
src/cpp/cooling_openacc.cpp \  
-o install/bin/cooling_acc_gpu_compare \  
-L"${HDF5_LIB}" \  
-Wl,-rpath,"${HDF5_LIB}" \  
-lhdf5_cpp -lhdf5 -lz -ldl -lm -lstdc++fs
```

In questo caso `nofma` viene specificato anche dentro `-gpu`, affinché la contrazione FMA sia disabilitata esplicitamente nel codice eseguito sulla GPU.

4 Profilo 2: benchmark dello speed-up

4.1 Obiettivo del profilo

Queste build servono esclusivamente a misurare i tempi. Non definiscono `USE_HDF5`, non includono header HDF5 e non collegano librerie HDF5. In esecuzione bisogna passare `none` come secondo argomento e 0 come intervallo di output.

4.2 CPU seriale ottimizzata

```
nvc++ -O3 -fast -tp=icelake -std=c++17 \  
src/cpp/cooling.cpp \  
-o install/bin/cooling_cpp_bench \  
-lstdc++fs
```

4.3 CPU OpenMP ottimizzata

```
nvc++ -O3 -fast -tp=icelake -std=c++17 \  
-mp \  
src/cpp/cooling_openmp.cpp \  
-o install/bin/cooling_omp_bench \  
-lstdc++fs
```

4.4 CPU OpenACC multicore ottimizzata

```
nvc++ -O3 -fast -tp=icelake -std=c++17 \  
-acc=multicore \  
-Minfo=accel \  
src/cpp/cooling_openacc.cpp \  
-o install/bin/cooling_acc_cpu_bench \  
-lstdc++fs
```

4.5 GPU OpenACC A100 ottimizzata

```
nvc++ -O3 -fast -std=c++17 \  
-acc=gpu \  
-gpu=cc80,fastmath \  
-Minfo=accel \  
src/cpp/cooling_openacc.cpp \  
-o install/bin/cooling_acc_gpu_bench \  
-lstdc++fs
```

5 Spiegazione dei flag

5.1 Flag comuni

- O3 abilita un livello elevato di ottimizzazione, incluse trasformazioni aggressive dei loop;
- std=c++17 seleziona lo standard C++17 richiesto dal codice;
- lstdc++fs collega il supporto GNU per `std::filesystem`; è mantenuto per compatibilità con lo stack del cluster;
- o file stabilisce il nome dell'eseguibile prodotto.

5.2 Flag di parallelizzazione

- mp abilita le direttive OpenMP e collega il runtime OpenMP di NVHPC;
- acc=multicore traduce le direttive OpenACC per eseguirle sui core della CPU;
- acc=gpu traduce le direttive OpenACC generando kernel per GPU NVIDIA;
- gpu=cc80 genera codice per compute capability 8.0, cioè l'architettura delle GPU NVIDIA A100 dei nodi Booster di Leonardo;
- Minfo=accel stampa a compilazione conclusa quali regioni e loop sono stati trasformati per il target parallelo. Non introduce overhead durante l'esecuzione.

5.3 Flag del profilo di confronto

- Kieee richiede un comportamento floating-point più rigoroso rispetto alle regole IEEE e limita alcune trasformazioni algebriche;
- Mnofma impedisce al compilatore host di contrarre una moltiplicazione e una somma in una singola FMA;
- gpu=cc80,nofma seleziona A100 e disabilita FMA anche nel codice dispositivo;
- DUSE_HDF5=1 attiva nel sorgente le sezioni protette da `#ifdef USE_HDF5`;
- I aggiunge la directory degli header HDF5;
- L aggiunge la directory delle librerie HDF5 al linker;
- Wl,-rpath,... registra nell'eseguibile il percorso delle librerie HDF5, così il loader può ritrovarle durante il job;
- lhdf5_cpp -lhdf5 collega rispettivamente le API C++ e C di HDF5;
- lz -ldl -lm collega le dipendenze usate dallo stack HDF5: compressione zlib, caricamento dinamico e libreria matematica.

5.4 Flag del profilo benchmark

- fast** abilita il gruppo di ottimizzazioni NVHPC consigliate per le prestazioni. Il contenuto preciso del gruppo può cambiare tra versioni del compilatore;
- tp=icelake** genera codice host specifico per le CPU Intel Ice Lake dei nodi Booster, consentendo l'uso delle relative istruzioni vettoriali;
- gpu=cc80,fastmath** seleziona A100 e usa implementazioni matematiche GPU più rapide e meno conservative. Può modificare gli ultimi bit dei risultati e non deve essere usato per stabilire la tolleranza del validator.

Nel profilo benchmark sono intenzionalmente assenti **-Kieee**, **-Mnofma**, **-DUSE_HDF5**, i percorsi HDF5 e tutte le librerie HDF5. In questo modo la misura non comprende scrittura dei campi né vincoli floating-point introdotti esclusivamente per il debug numerico.

6 Selezione del profilo negli script SLURM

Lo script `submit_cpp.sh` può selezionare l'eseguibile seriale corretto in base a un argomento passato a `sbatch`. Poiché i lanci di benchmark saranno più frequenti, **benchmark** è la modalità predefinita: se non si specifica alcun argomento viene eseguito `cooling_cpp_bench` senza output HDF5.

Prima di inviare il primo job bisogna creare le directory usate per log e risultati. In particolare, `logs/` deve esistere prima di `sbatch`, perché Slurm apre i file di standard output e standard error prima di eseguire lo script:

```
mkdir -p logs output
```

I possibili comandi di lancio sono:

Comando	Profilo	Eseguibile
<code>sbatch submit_cpp.sh</code>	benchmark	<code>cooling_cpp_bench</code>
<code>sbatch submit_cpp.sh benchmark</code>	benchmark	<code>cooling_cpp_bench</code>
<code>sbatch submit_cpp.sh compare</code>	confronto	<code>cooling_cpp_compare</code>

Nel profilo **benchmark** lo script passa **none** al posto del file HDF5 e usa un intervallo di output pari a zero. Nel profilo **compare** scrive invece `output/Cooling_cpp.h5` ogni 20 time step.

In modo analogo possono essere chiamati gli altri script per i vari casi: `submit_omp.sh`, `submit_acc_cpu.sh`, e `submit_acc_gpu.sh`.

7 Installazione dell'ambiente Python e validazione

Il validator `tools/validate_cooling_h5.py` usa i moduli Python `h5py` e `numpy`. Il modulo `python/3.11.7` di Leonardo fornisce l'interprete, ma non garantisce che questi pacchetti siano installati. Il progetto usa quindi un virtual environment chiamato `cooling_venv`, collocato nella directory principale del repository. L'installazione deve essere eseguita una volta sola.

Per creare l'environment si invia il job dalla directory principale del progetto:

```
sbatch submit_install_pyenv.sh
```

Con le tolleranze definite nello script, lanciare:

```
sbatch submit_validate.sh
```

Durante il confronto, `/step` viene verificato esattamente, mentre per `/field` viene applicata la condizione

$$|u_{\text{cand}} - u_{\text{ref}}| \leq \text{ATOL} + \text{RTOL} |u_{\text{ref}}|.$$